

Inteligência Artificial

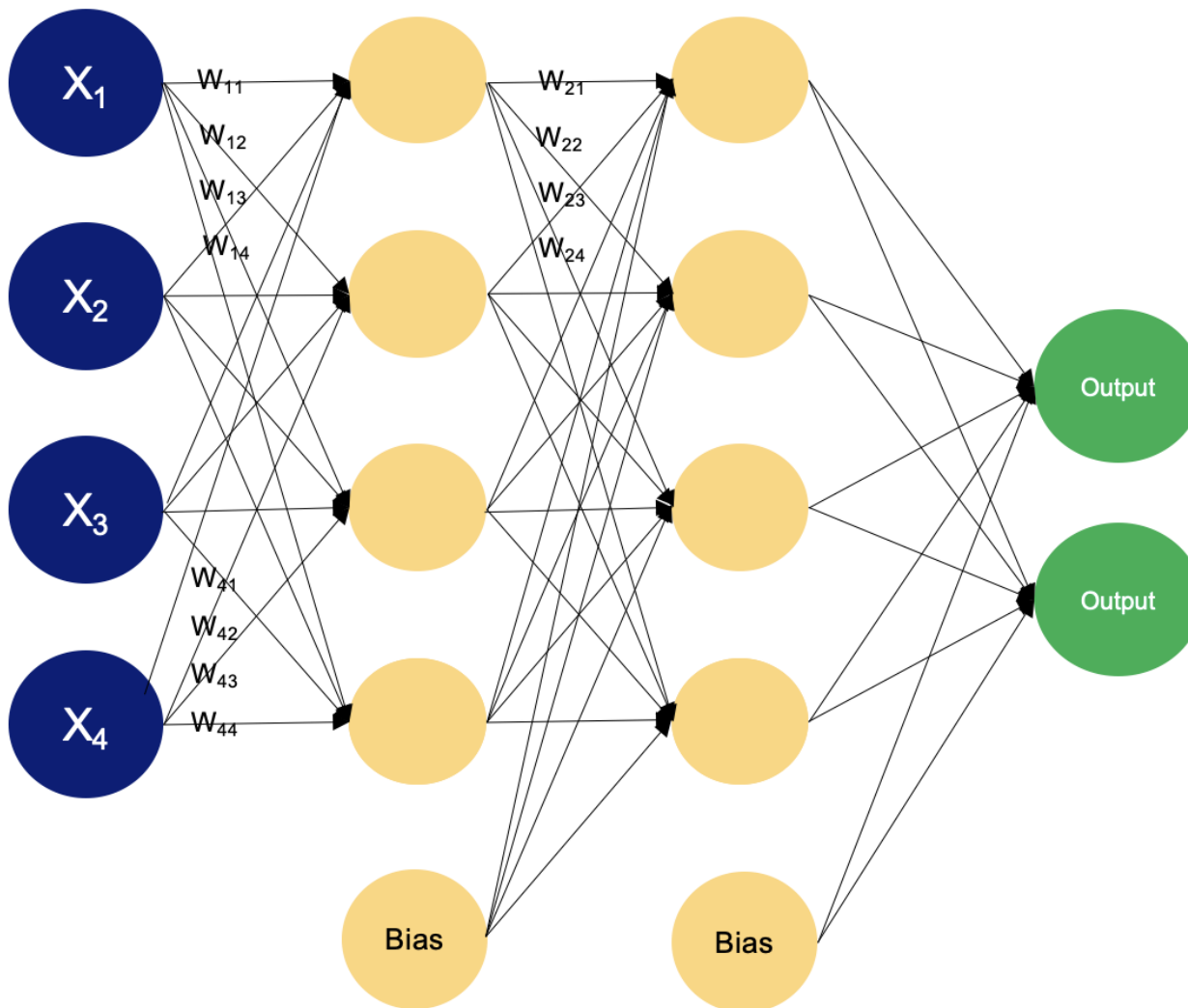
Redes Neurais e MLP Perceptron

Baseado no material do prof. Dr. Rogério de Oliveira (Mackenzie)

Prof. Dr. Ivan Carlos Alcântara de Oliveira

<https://orcid.org/0000-0002-6020-7535>

Inner layer | Hidden layers | Outer layer



Fonte: <https://www.datacamp.com/pt/tutorial/multilayer-perceptrons-in-machine-learning>.

Data da consulta: 06/11/2024.

Redes Neurais e MLP Perceptron

Introdução

- **Redes neurais de Arquitetura *Feed-Forward*.**
 - O psicólogo Frank Rosenblatt concebeu em 1957 o Perceptron, um **modelo matemático bastante simplificado de um neurônio biológico.**
 - **Perceptron Simples:**
 - **modelo básico de rede neural**
 - **uma única camada de saída**
 - **capaz de resolver apenas problemas linearmente separáveis.**

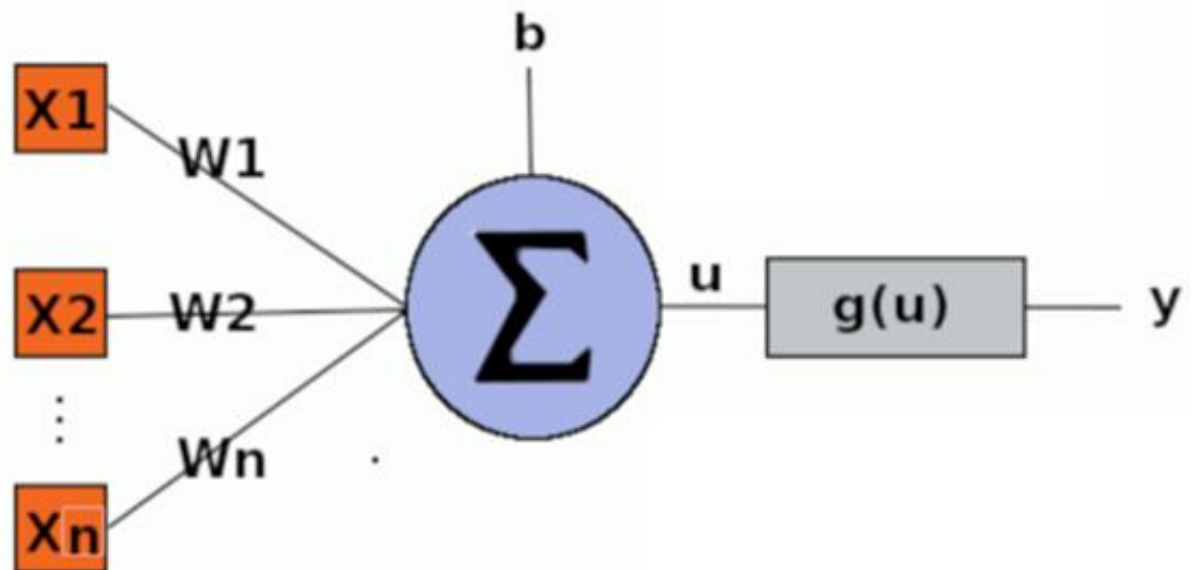
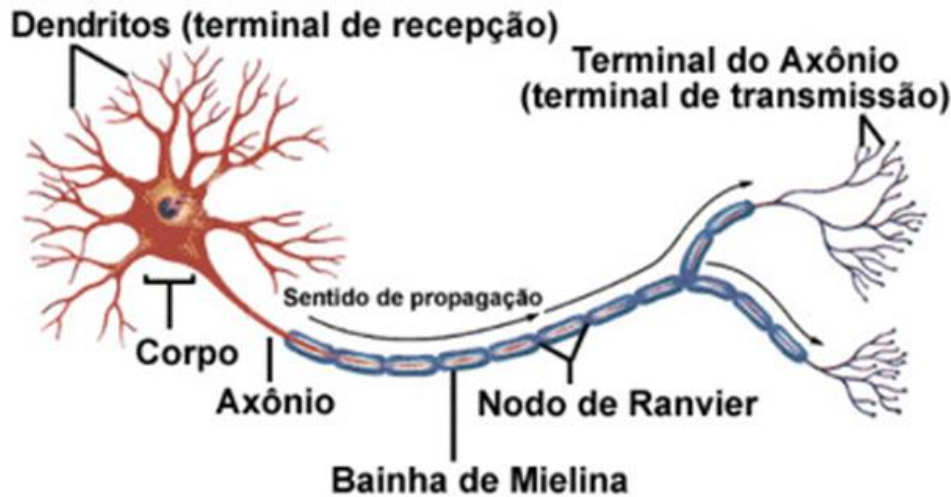
Redes Neurais e MLP Perceptron

Perceptron

- **Modelo empregado pela maior parte das redes neurais, incluindo as redes *deep learning* em TensorFlow, PyTorch etc.**
- O Perceptron é mais **adequado para problemas em que as classes são linearmente separáveis; caso contrário, seu desempenho pode ser limitado.**
- Esses modelos simples de neurônios **podem ser combinados para formarem redes de Perceptrons** com capacidade de resolver problemas mais complexos.

Redes Neurais e MLP Perceptron

Introdução



Redes Neurais e MLP Perceptron

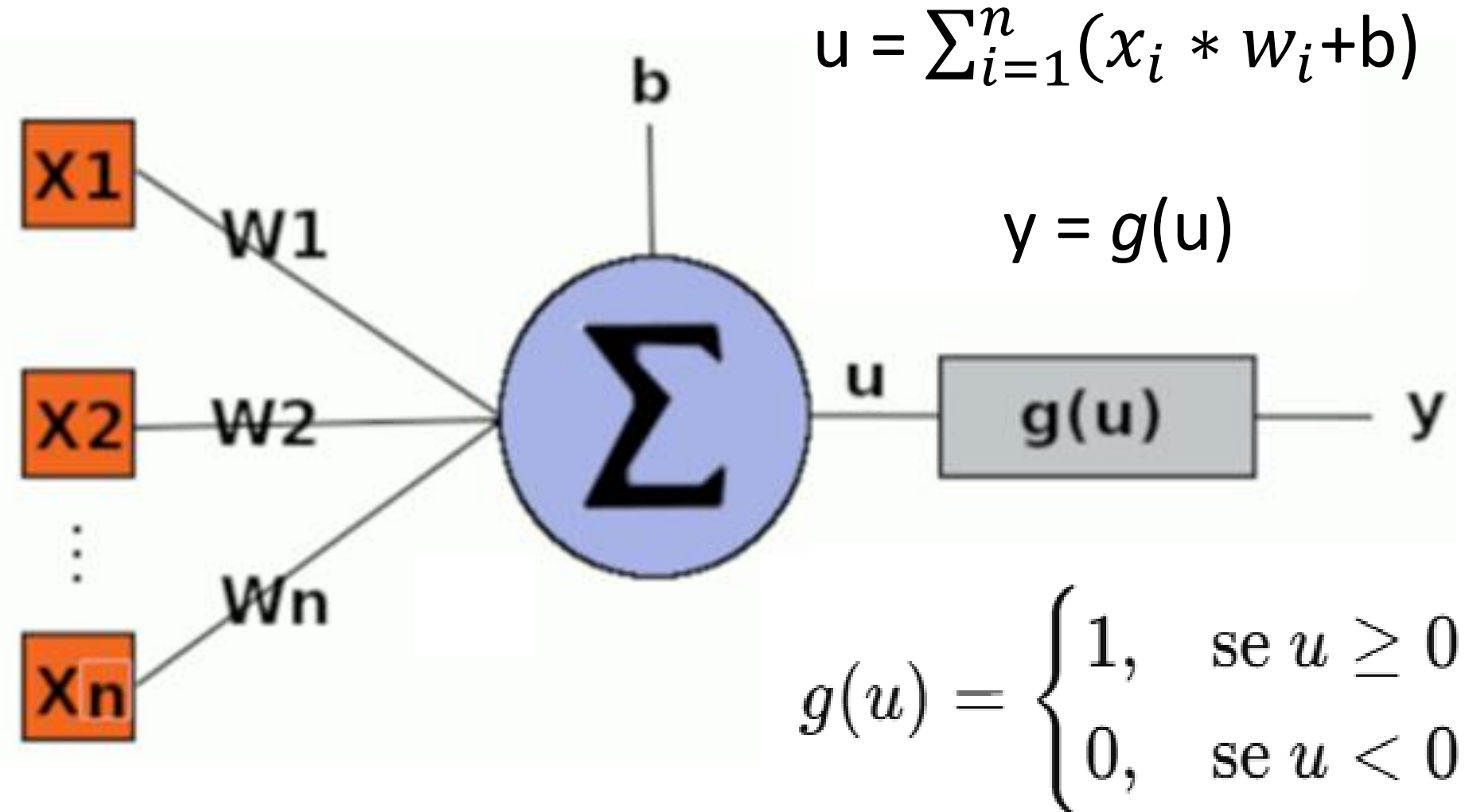
Perceptron

- Um neurônio artificial nada mais é do que um **artefato de software** que **faz uma combinação linear das entradas** e **aplica uma função de ativação** para produzir uma saída.
- O **treinamento do neurônio** é feito **ajustando-se os pesos w_n de acordo com o erro de predição (loss)** obtido para se estimar a saída $d \cong y$, onde d = saída desejada e y = saída calculada.

$$\min_w \sum \|d - y\|$$

Redes Neurais e MLP Perceptron

Introdução



Redes Neurais e MLP Perceptron

Perceptron – Estrutura Geral do Perceptron de uma camada única

- Um Perceptron calcula sua saída como:

$$y = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$

onde:

- x_i :entradas,
- w_i :pesos,
- b : *bias* (termo de limiar, também chamado de *offset* ou *threshold*),
- $f(\cdot)$:função de ativação.

Redes Neurais e MLP Perceptron

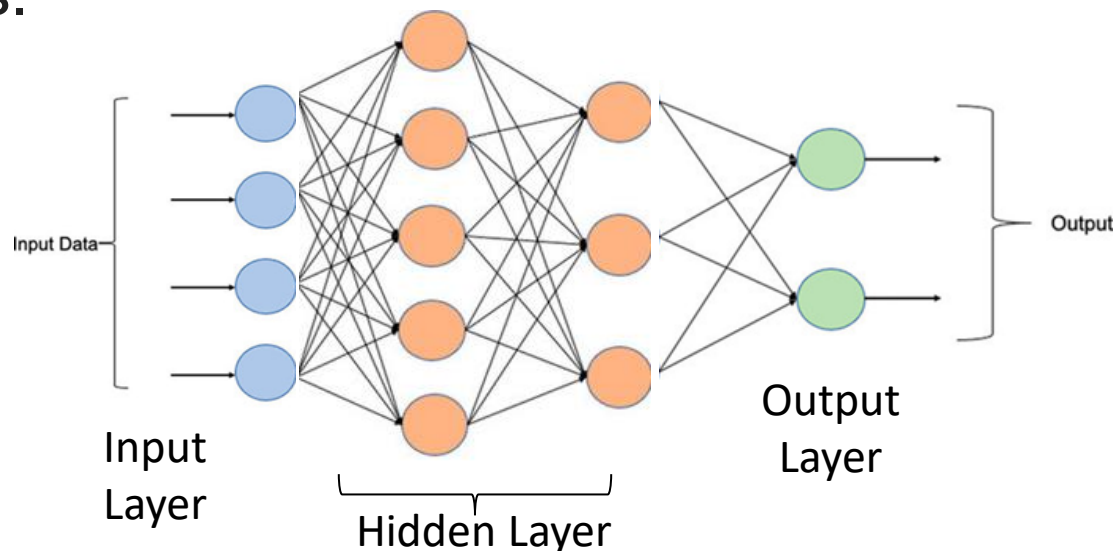
Perceptron

- É o modelo empregado pela maior parte das redes neurais, incluindo as redes *deep learning* em TensorFlow, PyTorch etc.
- Esses modelos simples de neurônios podem ser combinados para formarem redes de Perceptrons e resolverem problemas mais complexos.

Redes Neurais e MLP Perceptron

Introdução

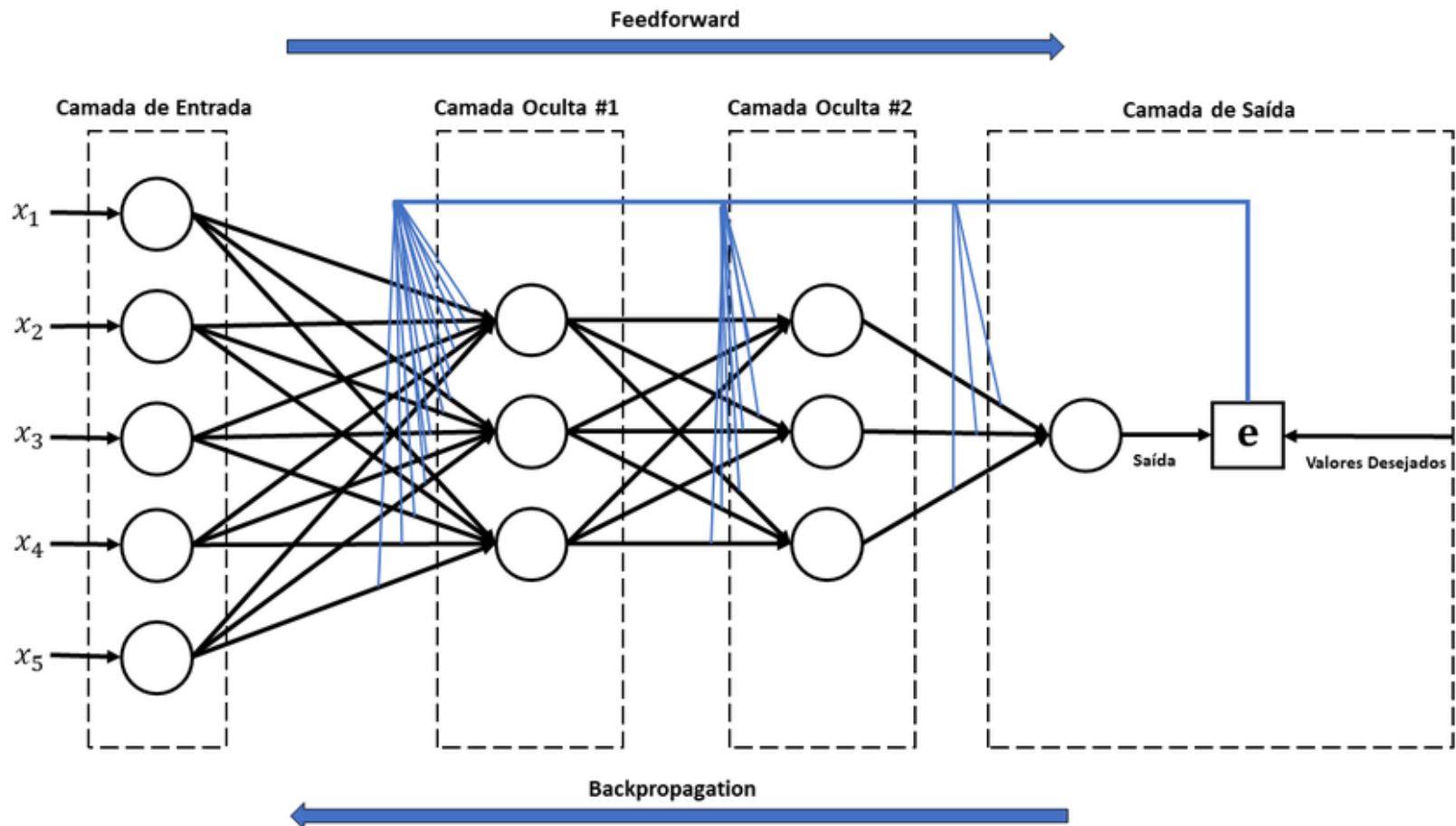
- **Redes neurais de Arquitetura *Feed-Forward*.**
 - **Multilayer Perceptron (MLP):** Extensão do perceptron simples. Possui uma ou mais camadas ocultas entre a camada de entrada e a de saída. Capaz de resolver problemas complexos, incluindo não linearmente separáveis.



Redes Neurais e MLP Perceptron

BackPropagation

- O backpropagation é o algoritmo de aprendizado central usado na MLP (*Multilayer Perceptron*) para ajustar os pesos e *biases* (w_0) da rede de modo a minimizar o erro entre a saída predita e o valor esperado.



Redes Neurais e MLP Perceptron

BackPropagation - Importante

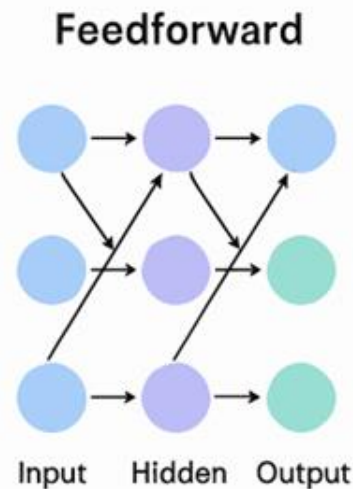
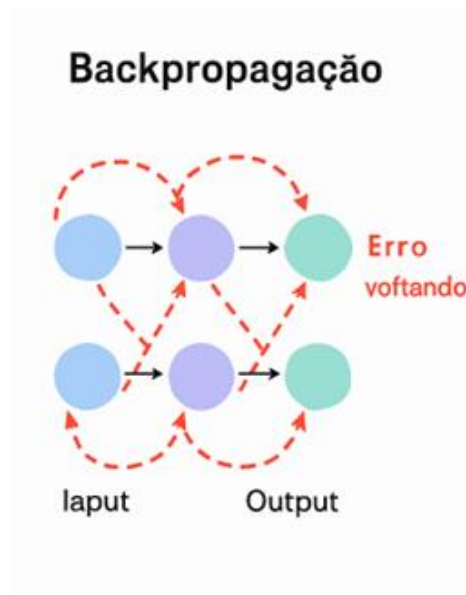
- ***Backpropagation* não é sobre a propagação dos dados.**
- **É sobre a propagação do erro** (ou seja, do sinal de ajuste dos pesos) durante o treinamento, não durante a execução normal da rede.
- Como funciona:
 - Primeiro é feita a propagação normal Feed-Forward (entrada até a saída).
 - Depois compara a saída gerada com a saída desejada e calcula o erro.
 - Então, o erro é propagado de trás para frente (da saída para as entradas) para ajustar os pesos de maneira eficiente (usando o gradiente descendente, por exemplo).

Redes Neurais e MLP Perceptron

BackPropagation - Importante

Importante:

- O backpropagation ajusta os pesos durante o treinamento, mas não cria ciclos na execução da rede.
- Durante o uso normal (inferência/predição), a rede continua sendo 100% Feed-Forward.



Redes Neurais e MLP Perceptron

BackPropagation - Metáfora

Metáfora (Analogia):

- ***Feed-Forward* é como dirigir do ponto A ao ponto B numa estrada.**
- **Backpropagation é como revisar o trajeto depois de chegar, vendo onde você errou para melhorar o caminho da próxima vez.**
- **Você não volta dirigindo para trás — você apenas corrige seu mapa para dirigir melhor na próxima viagem!**

Redes Neurais e MLP Perceptron

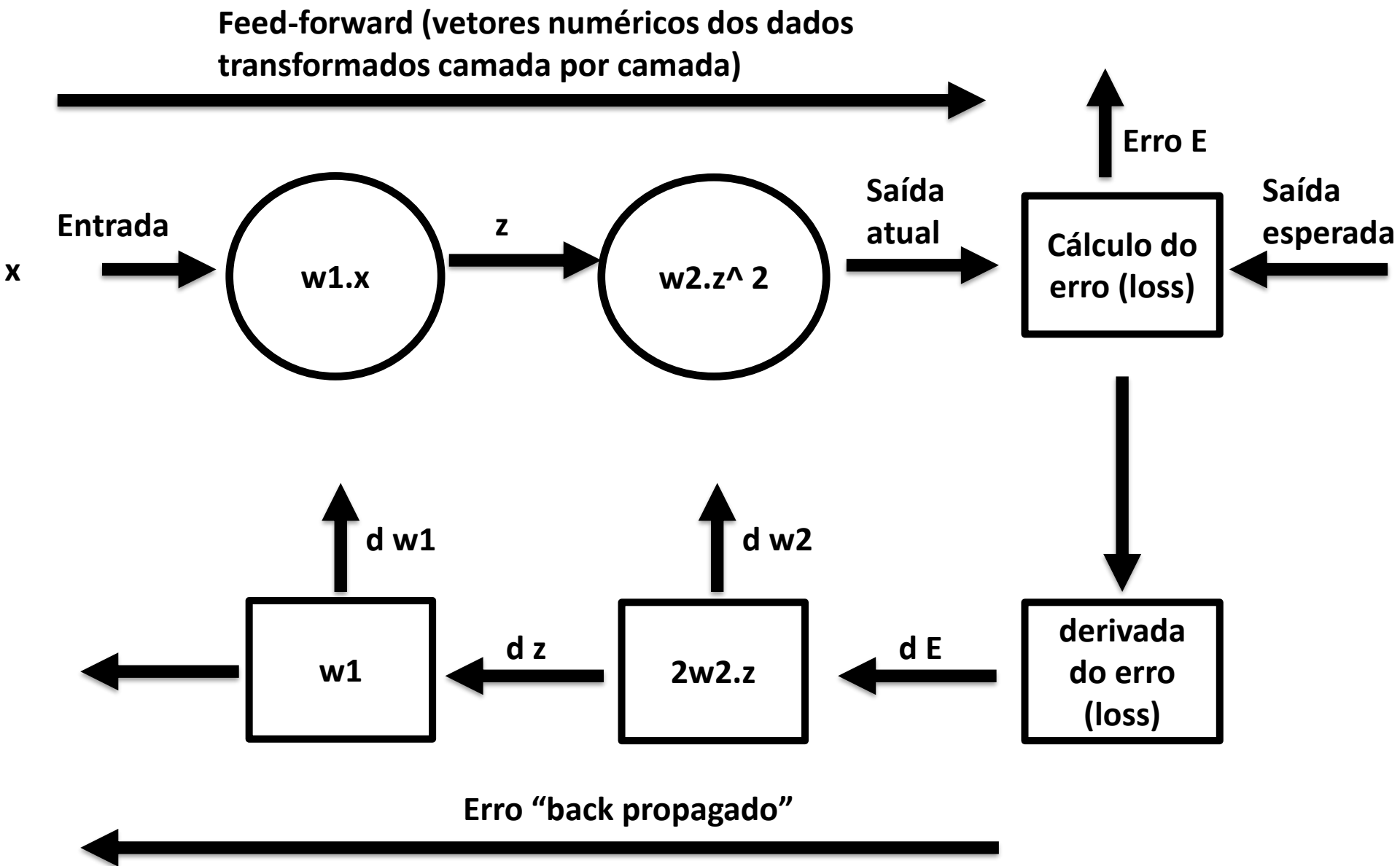
BackPropagation - Componentes

1. **Camadas da Rede Neural:** Cada camada (entrada, oculta e saída) com conexões (pesos) entre neurônios.
2. **Forward-Pass:** Setas que indicam o **fluxo dos dados da entrada para a saída** (em direção à frente), onde cada neurônio aplica um peso e uma função de ativação.
3. **Erro e Função de Perda (loss):** Na camada de saída, um **cálculo do erro** é mostrado, geralmente comparando a saída predita com o valor real.
4. **Backward Pass (Backpropagation):** Setas que indicam o **fluxo de volta do erro, da camada de saída para as camadas ocultas e de entrada**. Este fluxo mostra como o erro é "retropropagado" para ajustar os pesos de cada conexão.

Redes Neurais e MLP Perceptron

BackPropagation

- **O ajuste dos pesos é feito na direção contrária, a partir da derivada do erro obtido na camada de saída retropropagando o erro. Conhecido como backpropagation, ou algoritmo de retropropagação.**



Redes Neurais e MLP Perceptron

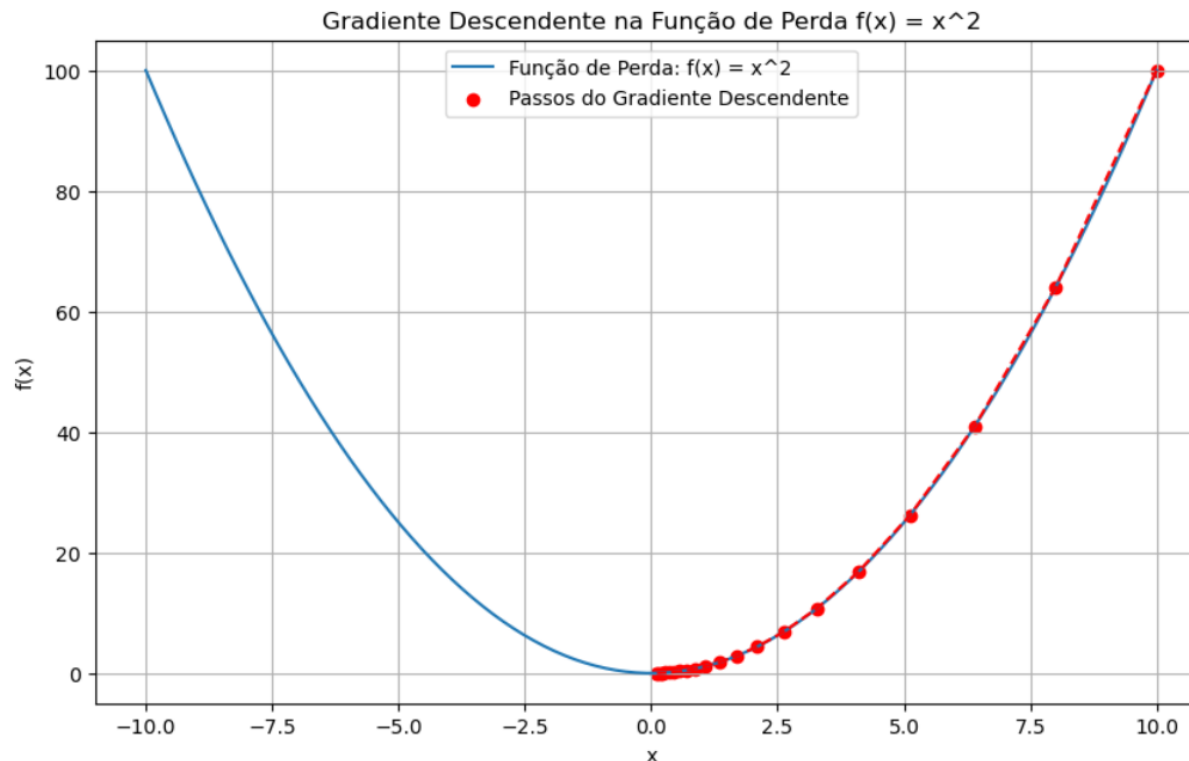
BackPropagation – Técnica de Otimização

- **As técnicas de otimização do Backpropagation são usadas para ajustar os pesos de uma rede neural durante o treinamento e tem a finalidade de melhorar a convergência (minizar a *loss function*), evitar mínimos locais e ajustar a velocidade de aprendizado de forma mais eficaz.**
- Dentre os otimizadores, temos:
 - **Gradiente Descendente**
 - **Gradiente Descendente Estocástico (SGD)**
 - Momentum
 - AdaGrad (Adaptive Gradient Algorithm)
 - RMSprop (Root Mean Square Propagation)
 - **Adam (*Adaptive Moment Estimation*), outros.**

Redes Neurais e MLP Perceptron

BackPropagation – Técnica de Otimização Descida do Gradiente (ou BGV, *Batch Gradient Descent*)

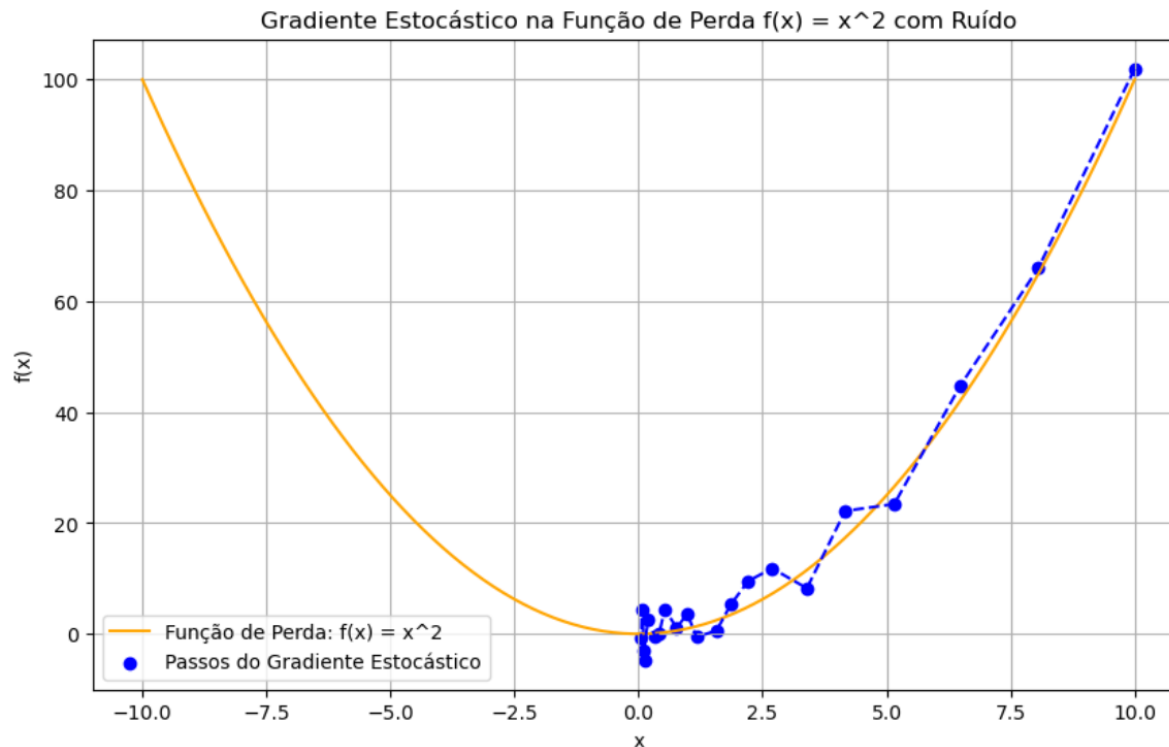
- **Descida do gradiente** é em geral um processo que **emprega as derivadas de uma função custo** (o erro) para repetidamente ajustar os pesos e minimizar o erro. **Emprega todo o conjunto de dados.**



Redes Neurais e MLP Perceptron

BackPropagation – **Gradiente Estocástico** (SGD, *Stochastic Gradient Descent*)

- **No Gradiente Estocástico é selecionado um pequeno lote/amostra(s) dos dados para fazer o ajuste dos pesos. O processamento é muito mais rápido e podemos definir o tamanho da amostra a ser empregada (`batch_size`).**



Redes Neurais e MLP Perceptron

BackPropagation – Técnicas de Otimização - Convergência

- **Problemas mais complexos, podem não ter a convergência do ajuste dos pesos garantida parando em um mínimo local.**
- **Esse é um dos grandes problemas do aprendizado de redes neurais.**

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron

A construção de modelos com redes MLP Multilayer Perceptron segue no geral os mesmos passos de qualquer modelo de aprendizado supervisionado.

- Exploração e Preparação dos Dados
- Separação dos Conjuntos de Treinamento e Teste
- Configuração do Modelo
- Treinamento do Modelo
- Avaliação do Modelo sobre o Conjunto de Teste
- Análise dos Resultados

Redes Neurais e MLP Perceptron

Introdução

- O **sci-kitlearn** tem alguns **modelos neurais**, o Perceptron e o MLP Perceptron.
- **Outros frameworks** de desenvolvimento de modelos neurais profundas (*Deep Learning*) são:
 - **Keras/TensorFlow** e
 - **PyTorch**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron

- Empregaremos o esquema geral de aprendizado supervisionado para implementar **modelos MLP Multilayer Perceptron com sci-kit learn para classificação**, porém MLP também pode ser utilizado para regressão.
- O **ajuste desses modelos é uma das tarefas mais desafiadoras da construção de modelos neurais.**
- Para construir bons modelos, **precisamos ajustar os hiperparâmetros e observar como eles afetam o aprendizado da rede** para a seleção dos melhores parâmetros de aprendizado.
- MLP é **implementado como uma sequência de camadas conectadas diretamente** (por isso, pode ser **sequencial em construção**), mas é um modelo de rede *feed-forward*.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Aplcando MLPClassifier - Exemplo

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    stratify=y, random_state=1234)

mlp = MLPClassifier(...)

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

print("Accuracy on training set:
{: .2f}".format(mlp.score(X_train, y_train)))

print("Accuracy on test set: {: .2f}".format(mlp.score(X_test,
y_test)))
```

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron

- A separação do conjunto de dados em **treino**, **validação** e **teste** é uma prática no treinamento de redes neurais (e em aprendizado de máquina em geral) **para garantir que o modelo seja:**
 - eficaz,
 - generalizável e
 - não enviesado.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron

- Exemplo de separação típica:
 - **treino: 60% a 80%,**
 - **validação: 10% a 20%,**
 - **teste: 10% a 20%.**
- Síntese da finalidade de cada conjunto:
 - **Treinamento: aprender/atualizar os pesos com base no erro (loss)**
 - **Validação: ajustar hiperparâmetros/evitar *overfitting***
 - **Teste: avaliar o desempenho final e real do modelo, após o treinamento e validação.**

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros – `hidden_layer_sizes`

Vamos explorar alguns parâmetros do nosso classificador MLP.

1. **`hidden_layer_sizes`**: indica a **quantidade de neurônios nas camadas internas da rede**. É uma tupla (). O tamanho da lista corresponde ao número de camadas ocultas e, cada posição, contem o número de neurônios da respectiva camada.

A quantidade de neurônios da camada de entrada são os atributos (features - X) e a quantidade na saída corresponde a variável alvo (y)

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **activation**

- 2. activation:** a função de ativação das camadas internas da rede entre 'logistic' (sigmoid), 'tanh', 'relu' e 'identity' (linear). O padrão é a 'relu'. Além dessas, temos 'softmax' para problemas de classificação.

A função será única para todas as camadas e elementos da rede.

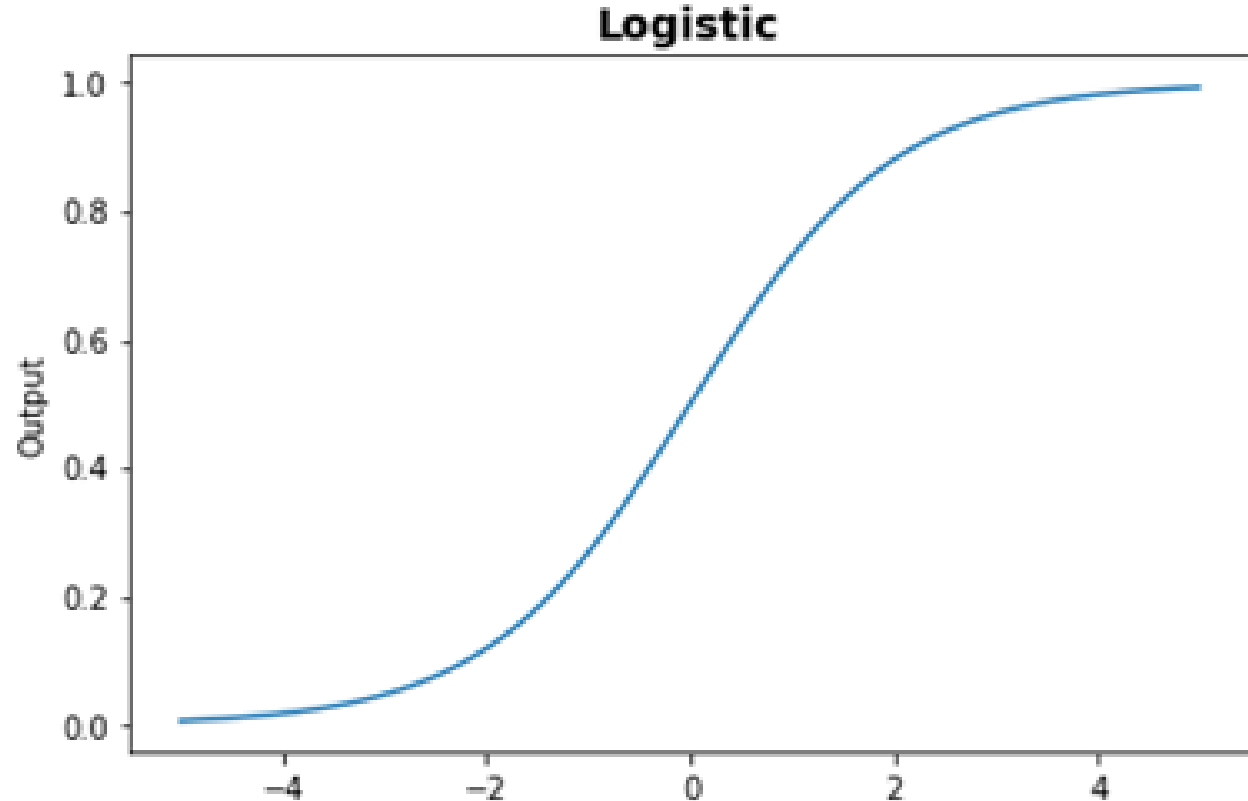
A função 'relu' não é adequada para camadas de saída, então, na camada de saída o **scikitlearn** utiliza automaticamente 'softmax' (classificação multiclasse), 'logistic' (duas classes - binárias) e 'identity' para regressão.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **activation** – Funções de Ativação

- **Funções de ativação logística ou sigmóide** (ou ainda *Soft Step*) são frequentemente usadas para problemas de classificação.

$$f(x) = \frac{1}{(1 + e^{-x})}$$

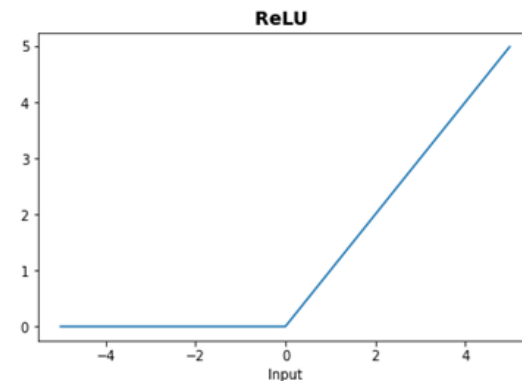


Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **activation** –
Funções de Ativação - Relu

- **Função Retificadora ReLu** (*Rectified Linear Unit*) é bastante eficaz para o aprendizado de redes profundas.
- Função preferencial para as camadas ocultas de *deep learning*.
- Nas camadas de saída, as funções logística e tanh são preferíveis por serem de fato classificadoras.
- Não devemos empregar funções ReLU na saída das redes.

$$f(x) = \max(0, x)$$

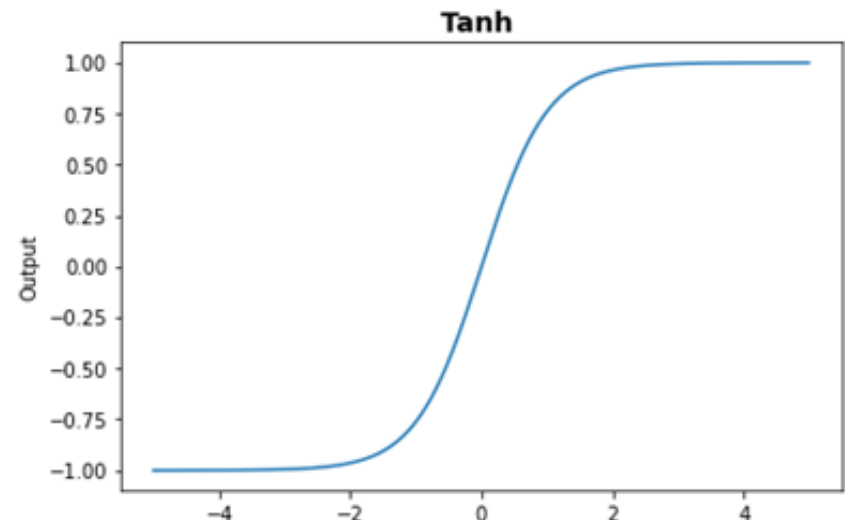


Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **activation** –
Funções de Ativação - Tangente Hiperbólica

- **Tangente Hiperbólica:** a tanh apresenta **melhor resultado quando as classes são mais claramente separáveis** e, assim como a função logística são **bastante empregadas em modelos neurais**.
- Para modelos *Deep Learning* elas são empregadas **principalmente nas camadas de saída**.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

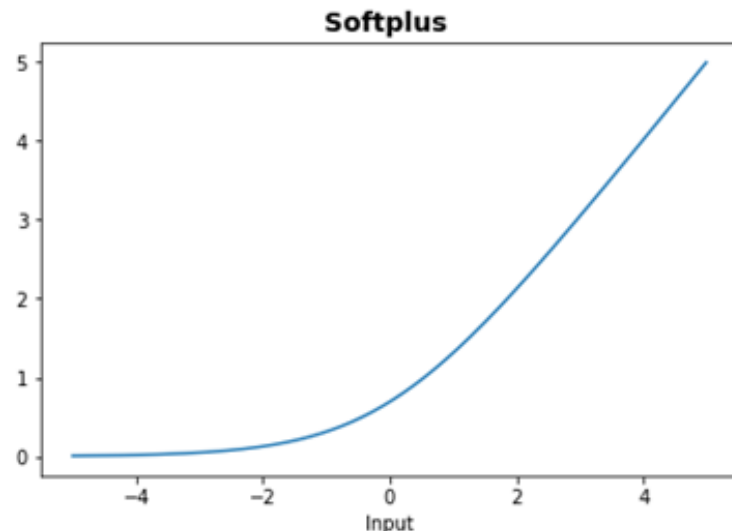


Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **activation** – Funções de Ativação

- **Função Softplus:** Ela é semelhante a ReLU mas diferenciável no ponto 0.
- A **Softplus** é diferenciável em qualquer ponto.
- Pode ser aplicada em problemas onde a ReLU apresenta **problemas** para a aplicação de certos otimizadores baseados na primeira derivada.

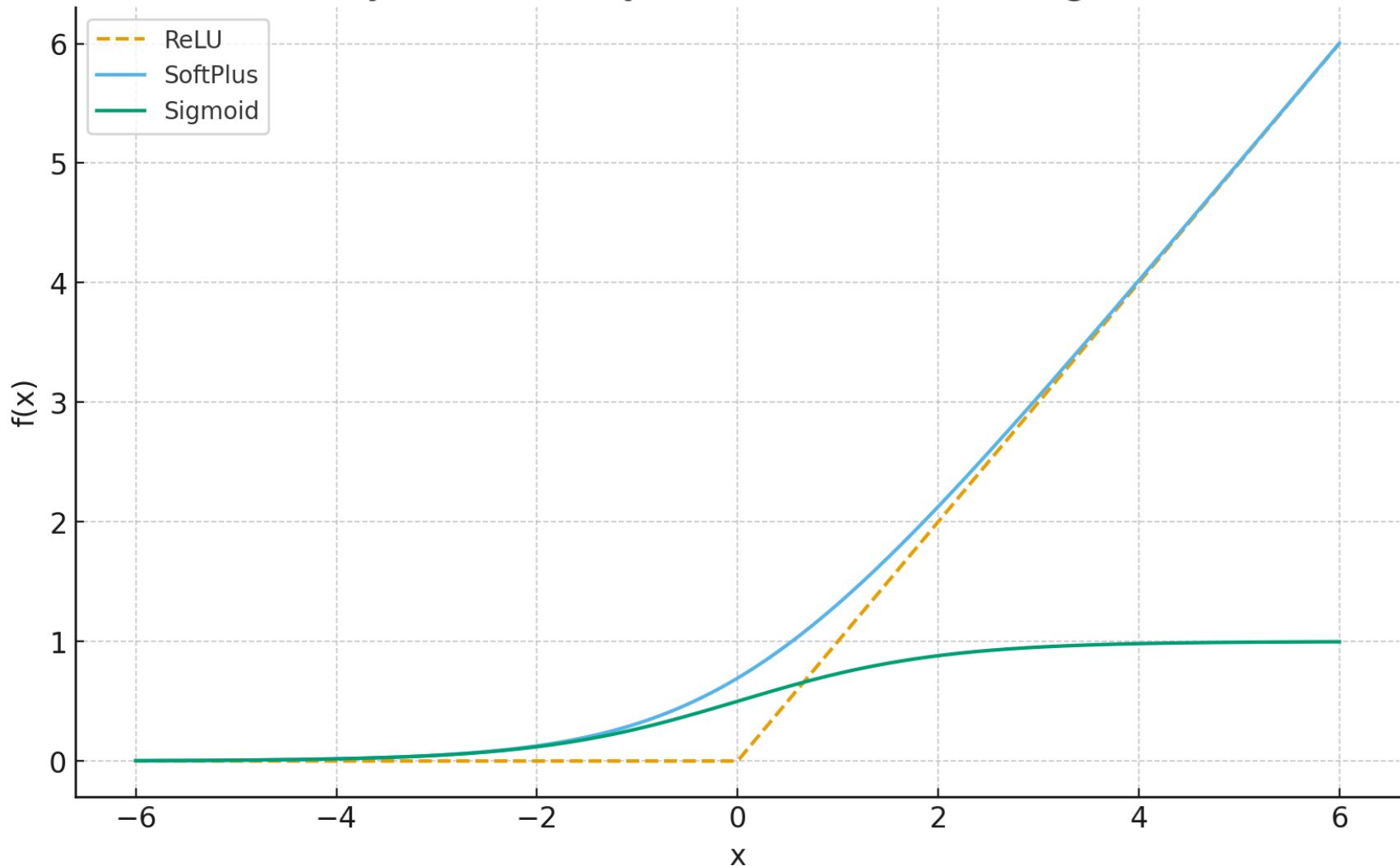
$$f(x) = \log(1 + e^x)$$



Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **activation** –
Funções de Ativação

Funções de Ativação: ReLU, SoftPlus e Sigmoid



Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros - **solver**

3. **solver**: define o **otimizador que será empregado**. O **adam** vem sendo o **mais empregado como padrão**, não só pelo `scikit_learn`, mas também por vários outros frameworks como o TensorFlow e PyTorch.
- **'sgd'** : gradiente estocástico.
 - **'adam'**: gradiente estocástico proposto por Diederik P. Kingma e Jimmy Lei Ba. **Funciona** muito **bem** em **conjuntos de dados relativamente grandes** (milhares de dados)
 - **'lbfgs'**: utilizado para encontrar mínimos de funções em problemas de otimização contínua que reduz significativamente os requisitos de memória. É uma **boa escolha** quando se trabalha com modelos menores e **conjuntos de dados de menor escala**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros – **learning_rate_init**

4. **learning_rate_init** (η_0): determina a quantidade de **ajuste aplicada aos pesos da rede em cada iteração** para **minimizar a função de erro** influenciando diretamente a velocidade e a qualidade da convergência do modelo. A taxa de aprendizado tem como **padrão** o valor **0.001**.

Ela por padrão é **constante ao longo de todo o treinamento**, mas isso **pode ser alterado** pelo parâmetro **learning_rate** (η) com os valores: **'constant'** (default), **'invscaling'** (com **power_t** - parâmetro adicional que controla a velocidade de decrescimento, ex.: **power_t = 0.5**) e **'adaptive'**.

A atualização dos pesos (w) é similar : $w = w - \eta \nabla L(w)$

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros – **validation_fraction**

- 5. **validation_fraction**: valor float entre 0 e 1, default=0.1 define a **fração dos dados de treinamento que será separada para validação durante o treinamento do modelo.**

Os conjuntos de validação são empregados durante o treinamento para ajustes dos parâmetros do modelo sendo essenciais no aprendizado de modelos neurais.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros – **max_iter**

6. **max_iter:** (int, default=200), número máximo de iterações do aprendizado e corresponde ao **número máximo de épocas de treinamento** que será empregado.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Parâmetros – max_iter

- 7. early_stopping: (bool):** define um **critério de parada do treinamento baseado na taxa de aprendizado (quando o treinamento não evolui após nr_iter_no_change)** e muitos outros parâmetros.

Para nossos propósitos esses parâmetros são suficientes, mas recomendo que você pesquise outras possibilidades na documentação do scikit-learn encontrada em: **https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Aplcando MLPClassifier - Exemplo

```
mlp = MLPClassifier( activation='tanh',  
    hidden_layer_sizes=(12, 12),  
    random_state=1234,  
    solver='adam',  
    learning_rate_init=0.001,  
    learning_rate='constant',  
    validation_fraction=0.1,  
    max_iter=300,  
    early_stopping=True,  
    n_iter_no_change=10)
```


Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos

- Os **pesos treinados do modelo** podem ser obtidos pela propriedade **coefs_** do modelo.

```
print("Pesos da Primeira Camada Oculta:")
```

```
print(mlp.coefs_[0])
```

```
print("Intercept da Primeira Camada:")
```

```
print(mlp.intercepts_[0])
```

- O **scikit-learn** diferencia os **coeficientes lineares** dos **intercepts**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos

- **Embora não tenhamos acesso direto ao 'significado' desses pesos eles desempenham algum papel em problemas de *transfer learning* e detecção de *features*.**
- Os pesos e intercepto em *transfer learning* permitem que o modelo adapte as features aprendidas em um domínio para serem aplicadas em outro, balanceando a retenção de características generalizadas com o ajuste para a nova tarefa.
- Isso torna o processo mais eficiente, economizando tempo e recursos no treinamento de um modelo completamente novo.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos

- Mas, **para um grande número de neurônios a inspeção direta dos valores parece bastante inadequada.**
- **Podemos empregar um heatmap para 'visualizar' as matrizes de pesos** e, assim, melhor explorar possíveis relações dos pesos e *features*.
- A operação de pesos com as entradas é basicamente um produto de matrizes a qual aplicamos em seguida a função de ativação.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos – Exemplo de Rede Neural com uma camada oculta

- Considere o **conjunto de dados Iris do scikit-learn**, carregado com **load_iris()**, contém **quatro atributos de entrada (*features*)**, que são medidas das características físicas das flores da espécie Iris. Esses atributos são:
 - 0. sepal length (comprimento da sépala - cm)
 - 1. sepal width (largura da sépala – cm)
 - 2. petal length (comprimento da pétala - cm)
 - 3. petal width (largura da pétala - cm)

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos – Exemplo de Rede Neural com uma camada oculta

- Essas quatro *features* são usadas para classificar **três espécies de flores íris: (0) Iris-setosa, (1) Iris-versicolor e (2) Iris-virginica**

iris setosa



petal

sepal

iris versicolor



petal

sepal

iris virginica

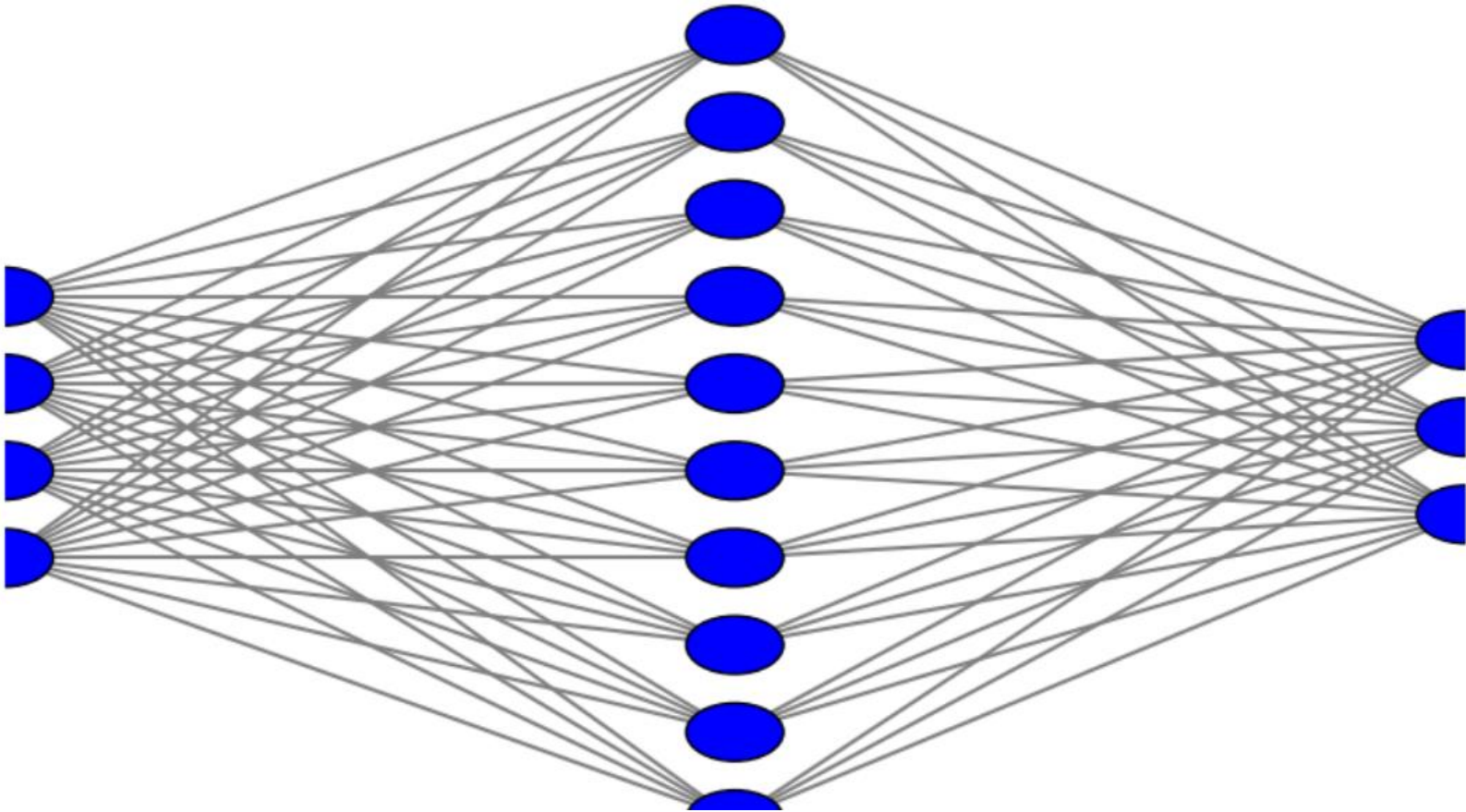


petal

sepal

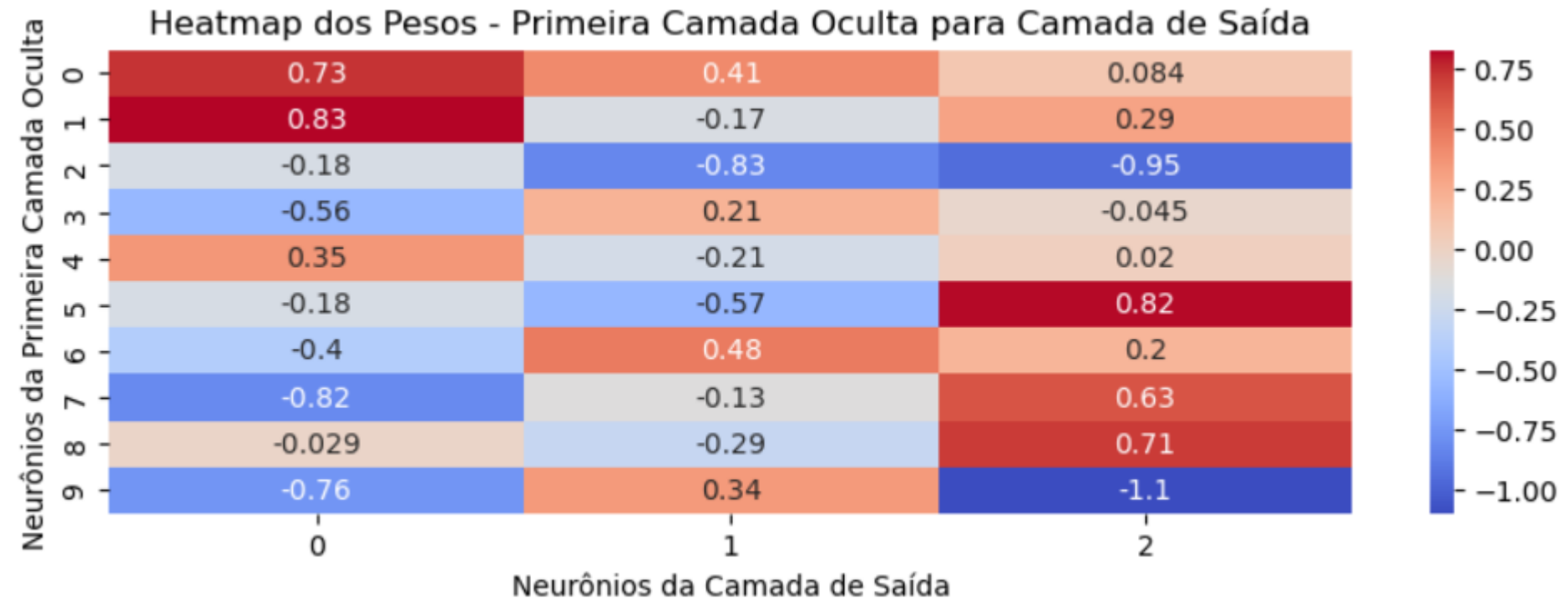
Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos – Exemplo de Rede Neural com uma camada oculta contendo 10 neurônios



Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos – Exemplo de Rede Neural com uma camada oculta – Matriz de Pesos - Saída



Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Pesos – Exemplo de Rede Neural com uma camada oculta – Matriz de Pesos - Saída

Interpretação das cores

-  **Vermelho forte:** peso positivo alto, significa que o neurônio oculto **contribui positivamente** para a ativação da respectiva classe.
-  **Azul forte:** peso negativo alto, significa que o neurônio oculto **inibe fortemente** essa classe.
-  **Branco ou cinza:** peso próximo de zero, representa **pouca ou nenhuma influência direta**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron - Fazendo o Tuning do Modelo: Hyperparameters

- Um dos grandes **desafios dos modelos neurais está em se fazer o *tunning* ou o ajuste de hiperparâmetros.**
- **Otimizá-los é selecionar os melhores hiperparâmetros para o aprendizado e fica a cargo do desenvolvedor do modelo.**

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Treinamento Incremental

- O **treinamento incremental** permite que o **modelo** seja **atualizado em pequenas porções de dados (*minibatches*)** ou **conforme novos dados são coletados, sem precisar recomeçar o treinamento do zero.**
- Disponível para alguns algoritmos em bibliotecas como **scikit-learn**, como modelos de Naive Bayes, **Perceptrons**, regressões logísticas e outros que suportam o método **partial_fit**.
- Esse método é particularmente útil em cenários onde:
 - **Grande Volume de Dados** e pouca memória.
 - **Novos Dados em Fluxo Contínuo** como monitoramento de redes sociais
 - **Modelos de Aprendizado Contínuo**: quando modelo precisa "aprender continuamente" e se adaptar as mudanças nos padrões dos dados ao longo do tempo.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Treinamento Incremental `fit` x `partial_fit`

- **`fit`**: treina o modelo em uma única chamada, processando todos os dados de uma vez. O `fit` recalcula todo o modelo a cada nova chamada. Adequado para situações em que todos os dados estão disponíveis e não precisam de ajuste incremental.
- **`partial_fit`**: permite treinar o modelo de maneira incremental. Cada chamada para `partial_fit` considera o estado atual dos pesos e realiza ajustes, permitindo treinar o modelo por "épocas manuais" de dados que chegam aos poucos ou não cabem na memória. **Não suporta `early_stopping`.**
- O método `partial_fit` pode ser utilizado no `MLPClassifier` e `MLPRegressor`.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Curva de Aprendizado

- **Acompanhar a Curva de Aprendizado ao longo do treinamento**, é importante nos modelos neurais que **apresentam longos períodos de treinamento** e um **grande número de hiperparâmetros de ajuste**.
- **O scikit-learn não apresenta uma função direta para isso ou para exibição dos resultados**. Então, para cada época é necessário armazenar os valores do:
 - **O loss**
 - **A acurácia de treino**
 - **A acurácia de Validação**

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Curva de Aprendizado

- **No `partial_fit`**, os passos para calcular cada métrica por etapa:
 - **Acurácia de Treino e validação:** usando `predict` e `accuracy_score`.
 - **Loss:** disponível pelo uso do atributo `loss_` do modelo.
- O **loss** é uma **função de perda que mede o erro**, onde:
 - **Loss Alto:** modelo está fazendo **previsões com grande erro**.
 - **Loss Baixo:** fazendo **previsões mais precisas**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Loss

Tipos comuns:

Regressão: Mean Squared Error (MSE) e Mean Absolute Error (MAE).

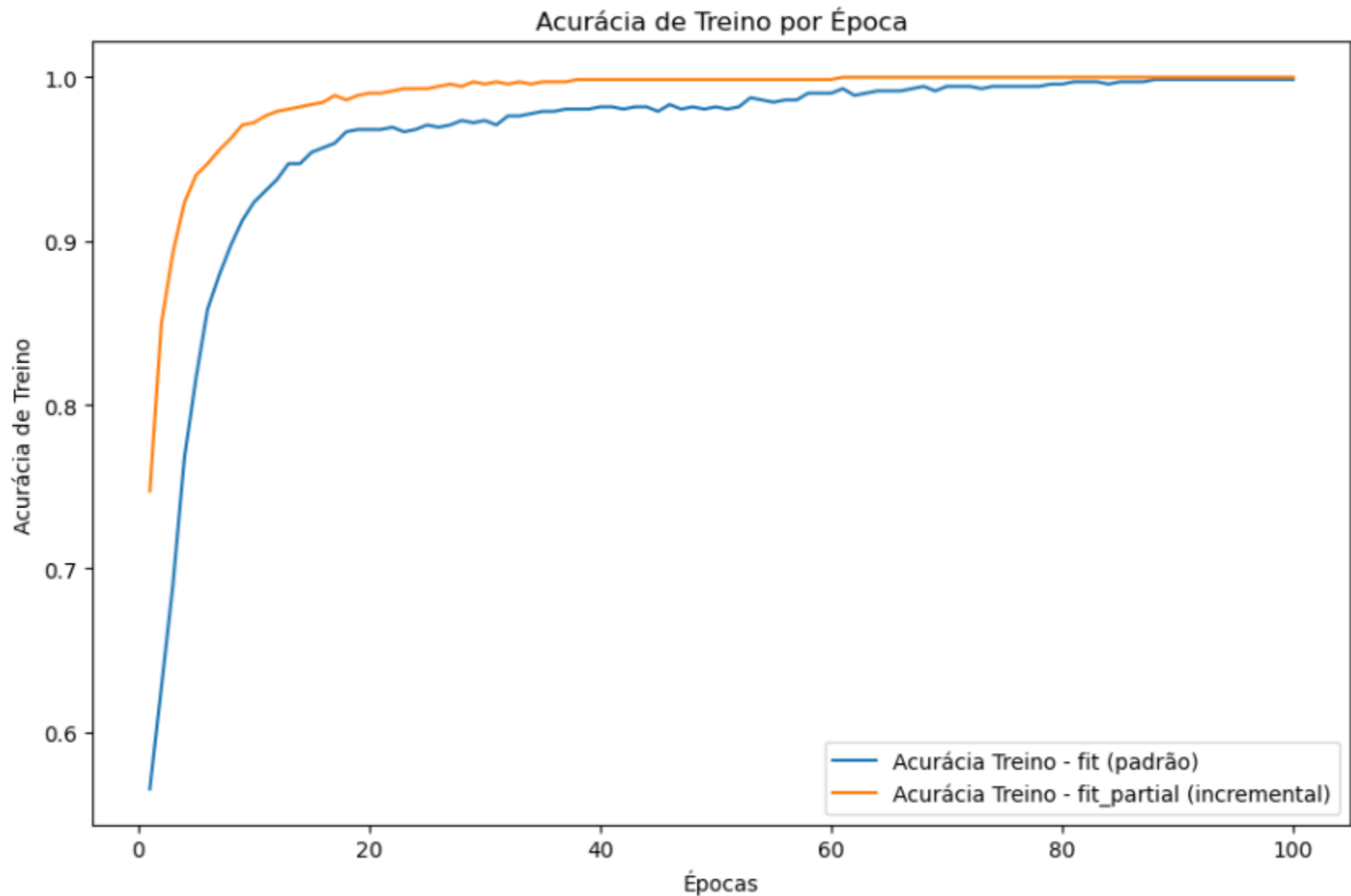
Classificação: Binary Cross-Entropy; Categorical Cross-Entropy; Hinge Loss.

No scikit-learn, o loss geralmente é configurado implicitamente ao escolher o tipo de modelo. onde

- No **MLPClassifier** a função de perda padrão é a **log-loss** (ou **entropia cruzada**).
- No **MLPRegressor**, a função de perda padrão é o **Mean Squared Error** (MSE).

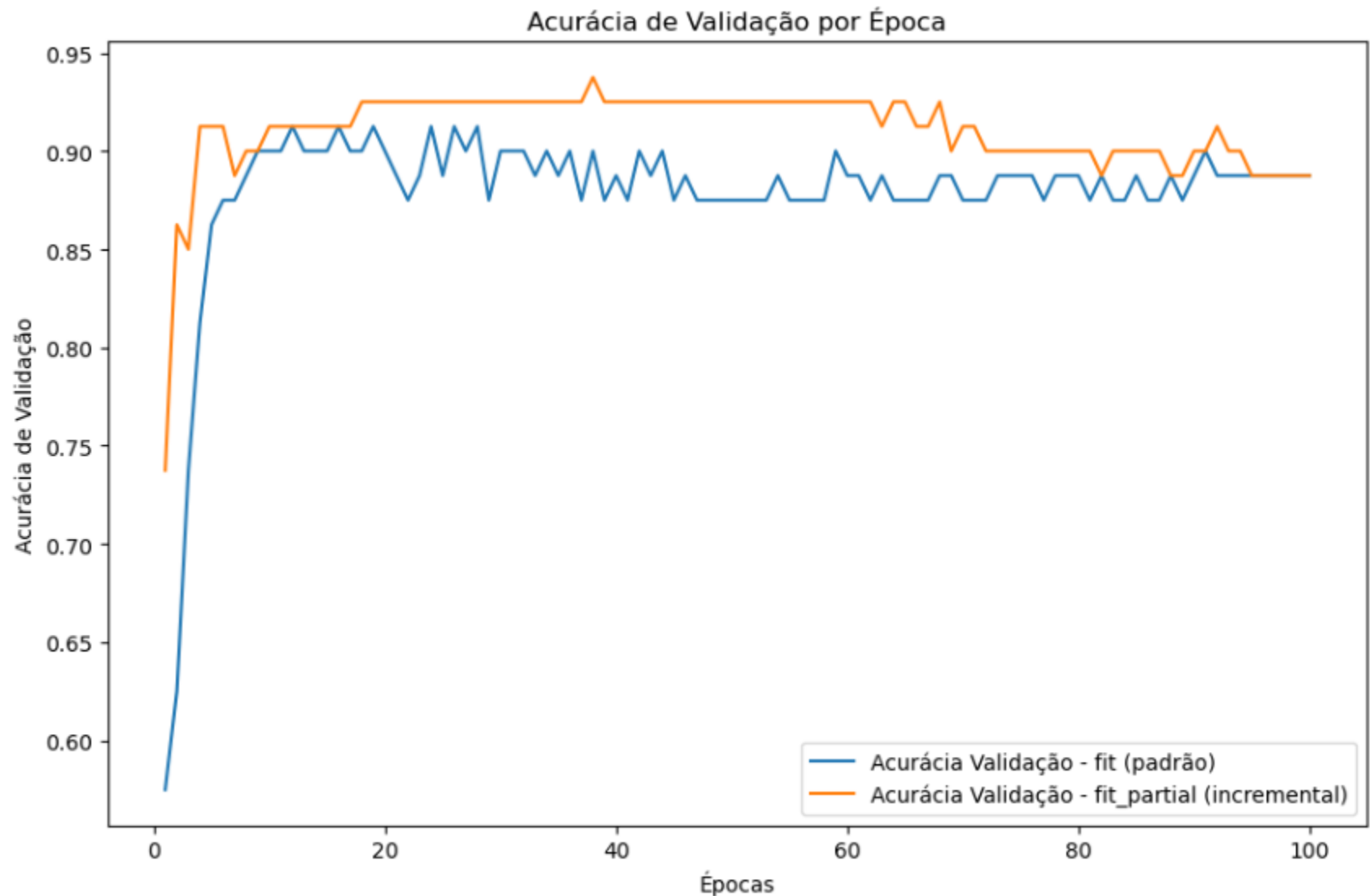
Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Acurácia de treino por época – fit e partial_fit



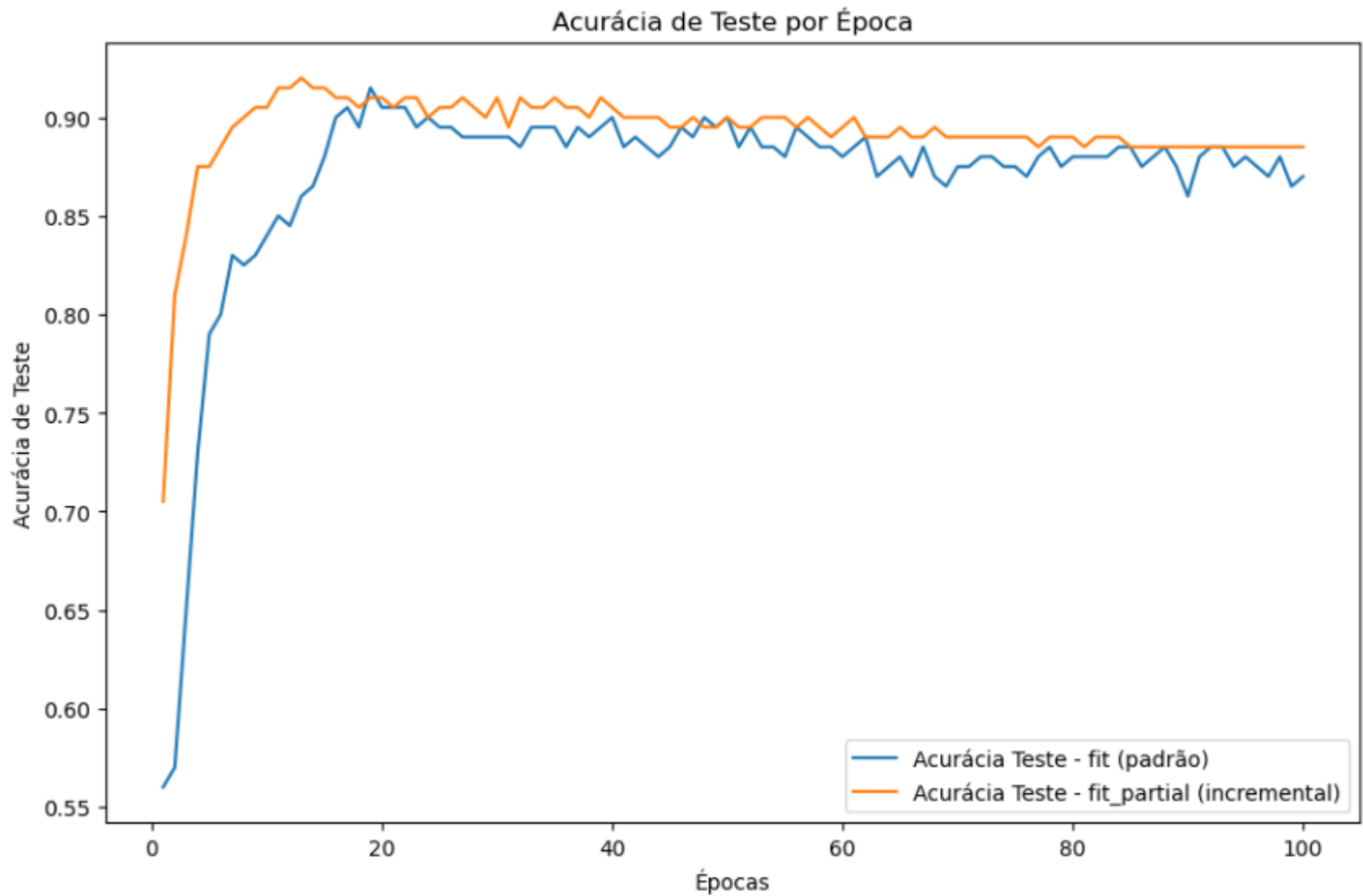
Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Acurácia de validação por época – fit e partial_fit



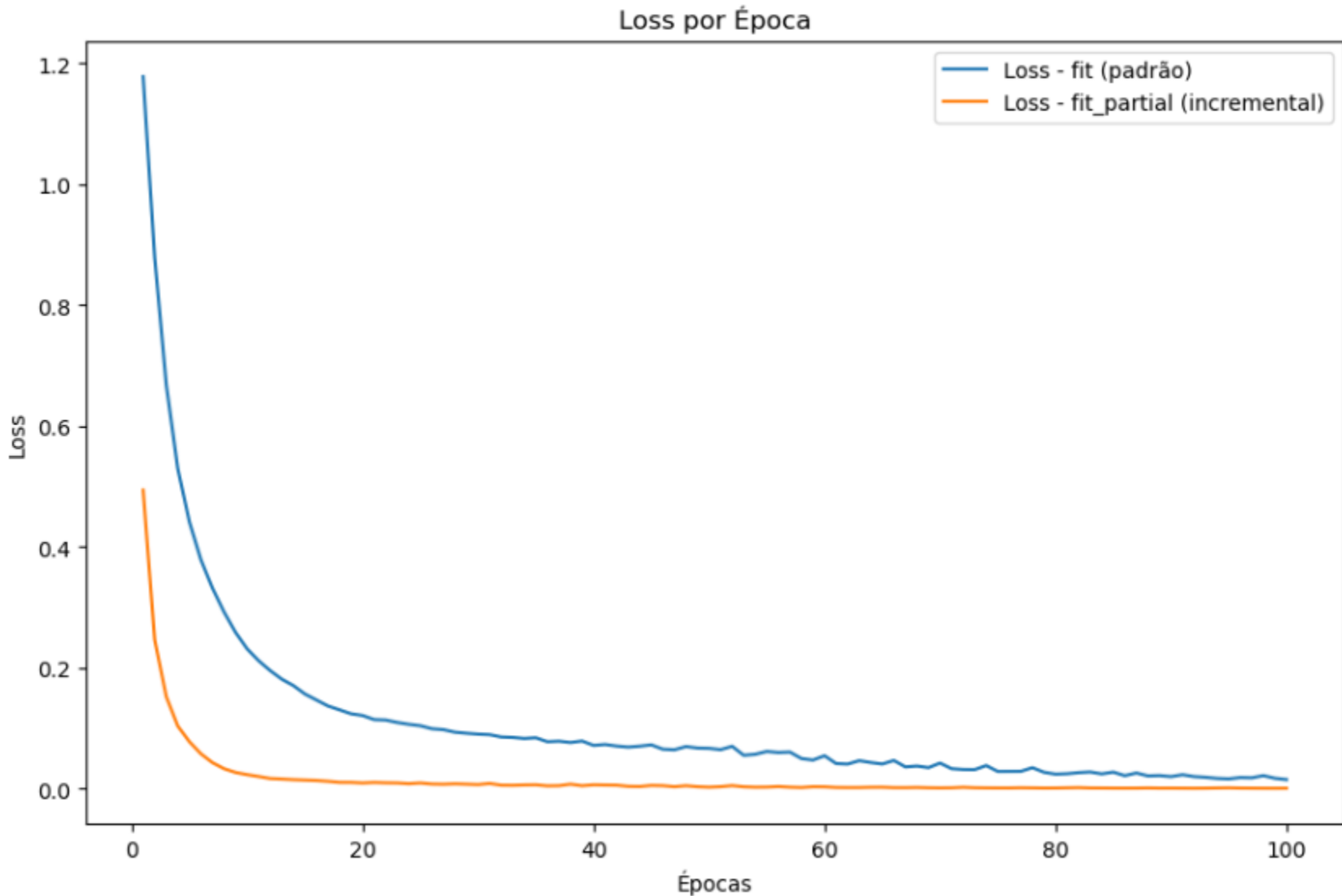
Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Acurácia de teste por época – fit e partial_fit



Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – loss por época – fit e partial_fit



Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Regularização e Dropout

- Em redes neurais profundas, a **regularização** e o **dropout** são **técnicas essenciais para combater o *overfitting***.
- No **scikit-learn**, o **MLPClassifier** e o **MLPRegressor** oferecem suporte à **regularização L2**, mas não têm suporte direto para regularização L1 nem dropout (TensorFlow ou PyTorch).
- No **scikit-learn**, pode-se configurar **L2** no **MLP** usando o **parâmetro alpha** (padrão: **0.0001 = 10^{-4}**). Varia de [**10^{-6}** a **1**]. Ajustado em faixa logarítmica **10^{-6} , 10^{-5} , 10^{-4} , 10^{-3} , 10^{-2} , 10^{-1} , 1,**
- **alpha**: controla a intensidade da regularização L2 e :
 - **Valores altos aumentam a regularização**, reduzindo a magnitude dos pesos.
 - **Valores baixos** têm um efeito mínimo, permitindo que o modelo se ajuste mais aos dados (mas com **maior risco de *overfitting***).

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Regularização e Dropout

- A regularização L2 **adiciona uma “penalidade”** quando os pesos do **modelo ficam muito grandes** fazendo o modelo evitar exagerar nos ajustes, **ajudando a não decorar os dados (*overfitting*)**.
- No scikit-learn, ela aparece como o parâmetro alpha, controlando o quanto essa penalização é forte.
- Exemplo de uso da regularização L2 com alpha:

```
from sklearn.neural_network import MLPClassifier
```

```
# Configurando o MLP com regularização L2
```

```
model = MLPClassifier(hidden_layer_sizes=(100,), alpha=0.01,  
random_state=42)
```

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Regularização e Dropout

- Para adicionar regularização extra, você pode:
- **Aumentar o valor de alpha:** Isso aumenta a regularização L2 e ajuda a controlar o overfitting.
- **Usar Early Stopping:** O MLPClassifier possui um parâmetro **early_stopping=True**, que interrompe o treinamento se a performance do modelo no conjunto de validação não melhorar após um certo número de épocas, ajudando a evitar **overfitting**.

Redes Neurais e MLP Perceptron

MLP Modelo Multilayer Perceptron – Regularização e Dropout

Exemplo com early_stopping:

```
model = MLPClassifier(  
    hidden_layer_sizes=(100,),  
    alpha=0.01,      # Regularização L2  
    early_stopping=True, # Parada antecipada  
    random_state=42 )
```

```
model = MLPRegressor(  
    hidden_layer_sizes=(100,),  
    alpha=0.01,      # Regularização L2  
    early_stopping=True, # Parada antecipada  
    random_state=42  )
```

Redes Neurais e MLP

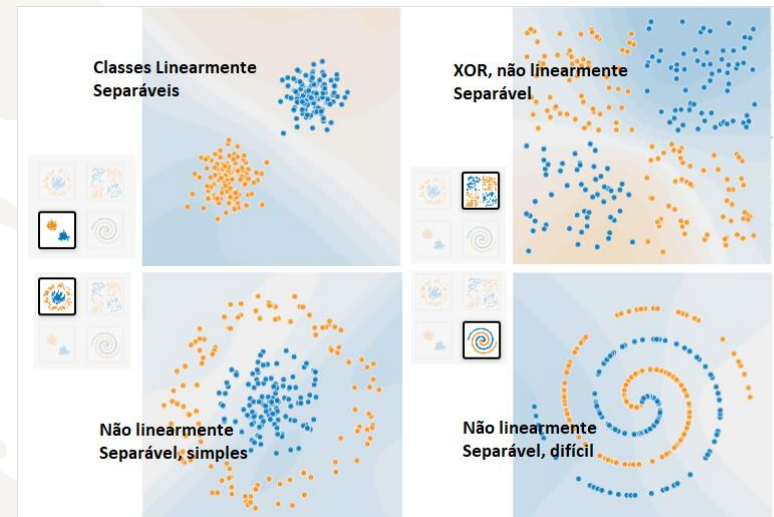
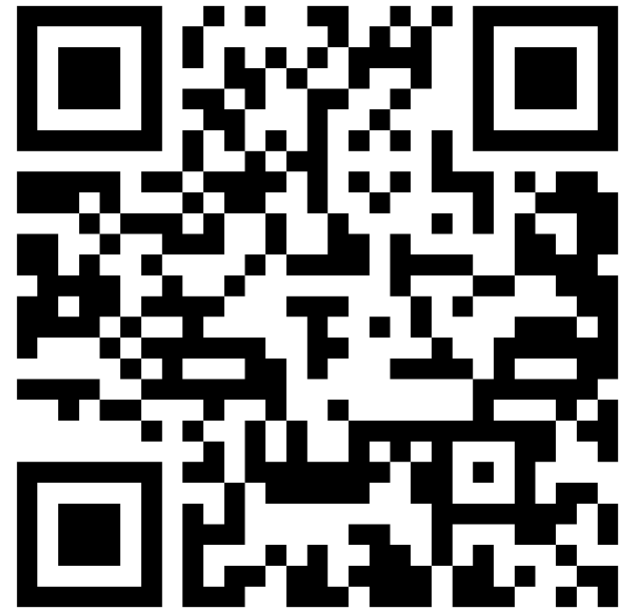
Perceptron

Perceptron – Experimentos sem programação

Acesse agora o site

<https://playground.tensorflow.org/>.

Aqui, alguns dos conjuntos de dados para classificação que você encontra no *playground*.



Redes Neurais e MLP Perceptron

Perceptron – Experimentos sem programação



Run/Pause

Epoch
000,046

Learning rate
0.03

Activation
Tanh

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

- X_1
- X_2
- X_1^2
- X_2^2
- $X_1 X_2$
- $\sin(X_1)$
- $\sin(X_2)$

+ - 2 HIDDEN LAYERS

+ -

4 neurons

+ -

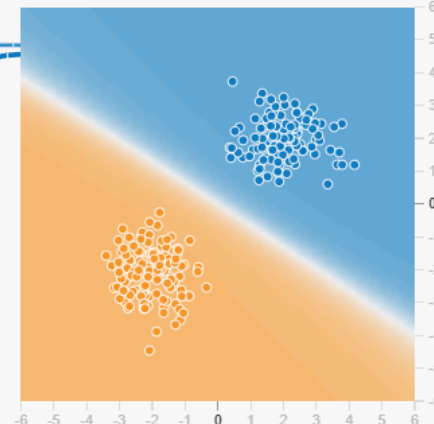
2 neurons

This is the output from one **neuron**. Hover to see it larger.

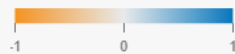
The outputs are mixed with varying **weights**, shown by the thickness of the lines.

OUTPUT

Test loss 0.002
Training loss 0.001



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

Redes Neurais e MLP Perceptron

Perceptron – Experimentos sem programação

Você poderá fazer uma série de ‘experimentos’ e observar o aprendizado de uma rede neural para diferentes problemas. Aqui algumas sugestões de exploração para você:

1. Comece configurando uma rede de 1 único neurônio e solucionando um problema linearmente separável. Observe o ajuste dos pesos e a redução do erro (Test e Training Loss).
2. Tente com o mesmo neurônio, solucionar um problema não linearmente separável. Veja que o erro não decresce e os dados não conseguem ser separados corretamente.

Redes Neurais e MLP Perceptron

Perceptron – Experimentos sem programação

3. Adicione agora uma ou duas camadas. Veja que agora um problema simples, mesmo não linear, pode ser rapidamente aprendido.
4. Experimente outros problemas mais difíceis (menos lineares). Você verá que para problemas mais complexos como pode ser necessário um grande número de elementos e camadas.

Bibliografia

BÁSICA:

- AGGARWAL, Charu C. **Artificial Intelligence: A Textbook**. New York: Springer: 2021.
- CHOLLET, François. **Deep Learning with Python, 2ed**. Shelter Island: Manning, 2021.
- GÉRON, Aurélien. **Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems**, 2 ed. Sebastopol: O'Reilly, 2019.

COMPLEMENTAR:

- GOODFELLOW, Ian; BENGIO, Yoshua, COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.
- RASCHKA, Sebastian; MIRJALILI, Vahid. **Python Machine Learning**. 3 ed. Birmingham: Packt, 2017.
- RUSSEL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3 ed. Upper Saddle River: Pearson, 2010.
- TAN, Pang-Ning; STEINBACH, Michael; KUMAR, Vipin. **Introduction to Data Mining**. 2 ed. Upper Saddle River: Pearson, 2018.
- VANDERPLAS, Jake. **Python Data Science Handbook**. Sebastopol: O'Reilly, 2017.

ADICIONAIS:

- FACELI, Katti et al. **Inteligência artificial: uma abordagem de aprendizado de máquina**. 2ª Ed. Rio de Janeiro: LTC- Livros Técnicos e Científicos, 2021.
- LUGER, George F. **Inteligência Artificial** - 6ª ed. São Paulo: Pearson Education do Brasil, 2015.
- RUSSELL, Stuart; NORVIG, Peter. **Inteligência artificial: Uma Abordagem Moderna** - 4ª. Ed. GEN LTC, 2022.